

Assignment: Simple Threads (Introduction to pthreads)

Up to this point, our programs have done one thing at a time (a single thread of execution). However, multithreaded programming is not a simple thing, so we're going to start slowly. To begin, we need to learn how to do multiple things simultaneously using POSIX threads (**pthreads**).

In this assignment, you will write a multi-threaded program that analyzes a large dataset of student coding logs. The goal is to count the total number of lines that contain the exact string "NO_DIFF".

Suggested Resources

- POSIX Threads (pthreads) tutorial: <https://computing.llnl.gov/tutorials/pthreads/>
- **pthread_create(3)**: Starts a new thread in the calling process. https://man7.org/linux/man-pages/man3/pthread_create.3.html
- **pthread_join(3)**: Waits for the specified thread to terminate and retrieves its return value. https://man7.org/linux/man-pages/man3/pthread_join.3.html
- **pthread_exit(3)**: Terminates the calling thread. https://man7.org/linux/man-pages/man3/pthread_exit.3.html
- **strcmp(3)**: Compares two strings. <https://man7.org/linux/man-pages/man3/strcmp.3.html>

*Note: You must compile programs using pthreads with the **-pthread** compiler flag. `gcc -pthread simple_threads.c -o simple_threads`*

Command-line arguments

Your program should accept exactly one command-line argument: **N**, the number of threads to create.

Example invocation for 4 threads: `./simple_threads 4`

Default behaviour

1. **Starter Code:** You are provided with `simple_threads.c`, which contains starter code that reads `massive_data.txt`. It parses the file and loads the diff text of each line into a globally accessible array of strings called `diff_array`. The total number of records loaded is stored in `total_records`.
2. **Workload Distribution:** Your program must split the array indices (0 to `total_records - 1`) as evenly as possible among the **N** threads.
3. **Thread Execution:** Each thread will iterate through its assigned range in the shared `diff_array` in a **read-only** manner. It will count the number of times the string exactly matches "NO_DIFF". Note that read-only is important here - we are not dealing with synchronization yet, and if

multiple threads try to access and modify the same memory, we can get race conditions that we aren't prepared to deal with.

4. **Returning Results:** Each thread must return its individual count of "NO_DIFF" occurrences back to the main thread (e.g., using `pthread_exit()` or by updating a designated field in an arguments `struct` passed to the thread). How you do this is up to you.
5. **Aggregation:** The main thread must wait for all child threads to finish using `pthread_join()`. It then sums all the sub-counts from the threads and prints the grand total of "NO_DIFF" lines to standard output.

Analyzing Performance with `time`

To see whether multi-threading has any effect on execution time, we will measure how long your program takes to run. The Linux `time` command measures the execution time of a given command.

You can use it by simply prepending `time` to your normal command:

```
time ./simple_threads 1
time ./simple_threads 2
time ./simple_threads 4
time ./simple_threads 8
```

When you run this, you will see output similar to the following:

```
real    0m0.045s
user    0m0.032s
sys     0m0.012s
```

- **real:** The actual “wall-clock” time from start to finish.
- **user:** The total amount of CPU time spent executing your code.
- **sys:** The total amount of CPU time spent in the operating system on behalf of your code.

Run your program with varying numbers of threads and observe how the `real` time changes. As you increase the number of threads up to the number of CPU cores available on your system, you should see the `real` time decrease, illustrating the power of parallel processing!